

CSE 425 — Fortran Finals Study Guide

From basics to tricky exam-style coding questions

PART 1 — CONCEPT CHEAT SHEET

1. Good Programming Style

- Keep logic simple and readable.
 - **Comment only what isn't obvious.** Don't write `! loop from 1 to 10` above `do i=1,10`. Do write `! loop to calculate nCr` above a binomial loop.
 - **Indent your code blocks** — it's expected and graded.
-

2. Declaring Variables

```
program main
  implicit none
  real :: a, b, total
```

- `implicit none` → **turns off** Fortran's old rule where variable type is guessed from its first letter. Always write this — it's the #1 habit examiners check.

`::` is the separator between type and variable name. Both are legal:

```
real :: a, breal a, b
```

-
- Without `implicit none` (old-style, avoid but know it for exam trick questions):
 - Names starting with `i, j, k, l, m, n` → automatically **integer**
 - Everything else → automatically **real**

Precision (KIND)

```
REAL(KIND = SELECTED_REAL_KIND(10,34)) :: variable_name
```

Gives at least 10 decimal digits of precision, range up to 10^{34} .

3. Data Types

Type	Example
integer	<code>integer :: number1</code>
real	<code>real :: a, b</code>
character	<code>character(len=20) :: test → test = 'Hello World'</code>
complex	<code>complex :: num1 → num1 = (3, 4)</code> means 3+4i
logical	<code>.true. / .false.</code>

Constants

integer, parameter :: constant1 = 5

real, parameter :: pi = 3.1428

`parameter` = value can never change once set.

Naming rules

Version	Rule
Fortran 77	1–6 characters, letters + digits only
Fortran 90	1–31 characters, letters + digits + underscore
	–

- Must **start with a letter**. `123abc` ❌
 - **Not case sensitive**: `abc = ABC`, `total = Total`
-

4. Operators & Precedence

Operator	Precedence (1 = highest)	Meaning
<code>**</code>	1	power
<code>*</code> , <code>/</code>	2	multiply, divide
<code>+</code> , <code>-</code>	3	add/unary+, subtract/unary-

Use `()` to force order.

5. Program Skeleton

```

program main
  implicit none
  real :: a, b, total
  a = 9
  b = 7
  total = a + b
  print *, 'The total is', total      ! same as write(*,*)
  write(*,*) 'The total is', total
end program main

```

- Everything sits between `program main` and `end program main`.
 - `print *, list` and `write(*,*) list` do the same thing.
-

6. Input / Output

```

read(stream, label) list
write(stream, label) list

```

- `stream` — usually `*` (default = screen/keyboard)
- `label` — format code, usually `*` (free format)
- `list` — comma-separated variables/strings to read or print

```

write(*,*) 'Enter a number:'
read(*,*) x

```

7. Logical Controls (if / else)

Comparison operators

Symbol	Word form	Meaning
<	<code>.lt.</code>	less than
<=	<code>.le.</code>	less or equal
==	<code>.eq.</code>	equal
>=	<code>.ge.</code>	greater or equal
>	<code>.gt.</code>	greater than
/=	<code>.ne.</code>	not equal

Logical combinators

`.not. .and. .or.`

Syntax

```
if (condition) then
  statement1
else if (condition) then
  statement2
else
  statement3
end if
```

Named (outer–inner) blocks — used in big programs for readability

```
outer: if (divisor_no /= 0) then
  inner: if (desired_no < 0) then
    write(*,*) 'Invalid input'
  else inner
    x = sqrt(desired_no)/divisor_no
  end if inner
end if outer
```

```
else outer
  write(*,*) 'Divided by zero issue'
end if outer
```

8. Loops (do)

```
do variable = start, stop [, step]
  statements
end do
```

- `exit` breaks out of the loop early.
- **do while** loop:

```
do while (condition)
  statements
end do
```

9. Intrinsic (built-in) functions (know these cold)

Function	Meaning
<code>abs(x)</code>	absolute value
<code>sqrt(x)</code>	square root
<code>sin(x), cos(x), tan(x)</code>	trig (radians)
<code>exp(x)</code>	e^x
<code>log(x)</code>	natural log
<code>int(x)</code>	truncate to integer
<code>aint(x)</code>	truncate toward zero, returns real
<code>anint(x)</code>	round to nearest, returns real
<code>mod(a,b)</code>	remainder of a/b — critical for even/odd, palindrome-style digit tricks

<code>len(str) /</code> <code>len_trim(str)</code>	length of string / length without trailing blanks
<code>trim(str)</code>	removes trailing blanks
<code>adjustl(str)</code>	removes leading blanks
<code>size(array)</code>	number of elements in array

10. Arrays

Basic declaration

```
real :: array_1(3)    ! 1D array, 3 values, indexed 1 to 3 by default
real :: array_2(3,3)  ! 2D array (matrix)
```

Custom index ranges (tricky — exam favorite)

```
real, dimension(-5:0, 0:5) :: array_1
! array_1(-5,0) is the first element, array_1(-5,5) ... array_1(0,5) is the last
```

Negative and custom start indices ARE allowed.

Whole-array math (no loop needed!)

```
real :: a(5), b(5), c(5)
c = a + b    ! adds element-by-element automatically
```

where (conditional array operation)

```
where (array > 0)
  array = array * 2
elsewhere
  array = 0
end where
```

Dynamic (allocatable) arrays

```
real, dimension(:), allocatable :: array_1
integer :: s
s = 100
```

```
allocate(array_1(s))
! ... use it ...
deallocate(array_1)
```

11. Functions

External function (defined after **end program**)

```
program main
  implicit none
  real :: avg, a, b, c, cal_avg
  read *, a, b, c
  avg = cal_avg(a, b, c)
end program main
```

```
real function cal_avg(x, y, z)
  real x, y, z, sum
  sum = x + y + z
  cal_avg = sum/3
end function cal_avg
```

Rules:

- Function must declare its own return type (**real function name(...)**).
- All arguments/local variables must be declared inside the function.
- **The function's name itself is assigned the return value.**
- **end function name** is mandatory.

Internal function (defined inside **program** using **contains**)

```
program main
  implicit none
  real :: x, r
  read*, x
  r = cube_root(x)
contains
  real function cube_root(x)
    implicit none
    real, intent(in) :: x
    cube_root = exp(log(x)/3.0)
  end function cube_root
```

end program main

`intent(in)` = tells the compiler this argument is read-only inside the function (protects it from being changed). **If you skip `intent(in)` and reassign `x` inside the function, it can silently corrupt behavior — classic trick question.**

12. Subroutines

Like functions, but **don't return a value** — instead they modify their arguments directly.

```
program swap_number_main
  implicit none
  real :: num1, num2
  read(*,*) num1, num2
  call swap_num(num1, num2)
  write(*,*) num1, num2
contains
  subroutine swap_num(x_first, y_second)
    real :: x_first, y_second, temp
    temp = x_first
    x_first = y_second
    y_second = temp
  end subroutine swap_num
end program swap_number_main
```

Key differences from functions:

- Called with `call name(args)`, not `x = name(args)`
 - No `= name` return-value line needed
 - Changes to the arguments **persist back** in the caller
-

13. File I/O

```
open(12, file = 'example.dat', status = 'new')
do i = 1, n
  write(12,*) radius_array(i), area_array(i)
end do
close(12)
```

- 12 is just an ID number you choose to refer to the open file.
 - status can be 'new', 'old', 'replace', etc.
 - Always close() the file when done.
-

PART 2 — PRACTICE QUESTIONS (Easy → Trickiest)

LEVEL 1 — Basics

Q1. Add two numbers and print the result.

```
program add_numbers
  implicit none
  real :: a, b, total
  write(*,*) 'Enter two numbers:'
  read(*,*) a, b
  total = a + b
  write(*,*) 'The total is', total
end program add_numbers
```

Q2. Compute the area of a circle given the radius.

```
program circle_area
  implicit none
  real, parameter :: pi = 3.1416
  real :: radius, area
  write(*,*) 'Enter radius:'
  read(*,*) radius
  area = pi * radius**2
  write(*,*) 'Area is', area
end program circle_area
```

Q3. Check if a number is even or odd.

```
program even_odd
```

```

implicit none
integer :: n
write(*,*) 'Enter a number:'
read(*,*) n
if (mod(n,2) == 0) then
    write(*,*) n, 'is even'
else
    write(*,*) n, 'is odd'
end if
end program even_odd

```

LEVEL 2 — Loops, if-else chains, arrays

Q4. Calculate factorial of a number using a do loop.

```

program factorial
    implicit none
    integer :: n, i, fact
    write(*,*) 'Enter a number:'
    read(*,*) n
    fact = 1
    do i = 1, n
        fact = fact * i
    end do
    write(*,*) 'Factorial is', fact
end program factorial

```

Q5. Assign a letter grade based on score (nested if-else).

```

program grade
    implicit none
    real :: score
    write(*,*) 'Enter score:'
    read(*,*) score
    if (score >= 90) then
        write(*,*) 'Grade: A'
    else if (score >= 80) then
        write(*,*) 'Grade: B'
    else if (score >= 70) then
        write(*,*) 'Grade: C'
    else
        write(*,*) 'Grade: F'
    end if
end program grade

```

```
end if
end program grade
```

Q6. Find the sum, average, and largest value in an array of 5 numbers.

```
program array_stats
  implicit none
  real :: arr(5), total, avg, largest
  integer :: i
  write(*,*) 'Enter 5 numbers:'
  read(*,*) (arr(i), i = 1, 5)

  total = 0
  largest = arr(1)
  do i = 1, 5
    total = total + arr(i)
    if (arr(i) > largest) largest = arr(i)
  end do
  avg = total / 5

  write(*,*) 'Sum =', total
  write(*,*) 'Average =', avg
  write(*,*) 'Largest =', largest
end program array_stats
```

Q7. Check whether a number is prime.

```
program is_prime
  implicit none
  integer :: n, i
  logical :: prime
  write(*,*) 'Enter a number:'
  read(*,*) n
  prime = .true.

  if (n < 2) then
    prime = .false.
  else
    do i = 2, n - 1
      if (mod(n, i) == 0) then
        prime = .false.
        exit
      end if
    end do
  end if
```

```

        end do
    end if

    if (prime) then
        write(*,*) n, 'is prime'
    else
        write(*,*) n, 'is not prime'
    end if
end program is_prime

```

LEVEL 3 — Functions & Subroutines

Q8. External function: check if a year is a leap year.

```

program leap_year_main
    implicit none
    integer :: year
    logical :: is_leap
    write(*,*) 'Enter a year:'
    read(*,*) year
    if (is_leap(year)) then
        write(*,*) year, 'is a leap year'
    else
        write(*,*) year, 'is not a leap year'
    end if
end program leap_year_main

logical function is_leap(y)
    integer :: y
    is_leap = (mod(y,4) == 0 .and. mod(y,100) /= 0) .or. (mod(y,400) == 0)
end function is_leap

```

Q9. Subroutine: swap two numbers AND double them at the same time.

```

program swap_double
    implicit none
    real :: a, b
    read(*,*) a, b
    call process(a, b)
    write(*,*) 'After swap+double:', a, b
contains
    subroutine process(x, y)

```

```

    real :: x, y, temp
    temp = x
    x = y * 2
    y = temp * 2
end subroutine process
end program swap_double

```

Q10. Internal function with `intent(in)`: compute GCD of two numbers.

```

program gcd_main
  implicit none
  integer :: a, b, result
  read(*,*) a, b
  result = find_gcd(a, b)
  write(*,*) 'GCD is', result
contains
  integer function find_gcd(x, y)
    implicit none
    integer, intent(in) :: x, y
    integer :: p, q, temp
    p = x
    q = y
    do while (q /= 0)
      temp = q
      q = mod(p, q)
      p = temp
    end do
    find_gcd = p
  end function find_gcd
end program gcd_main

```

Q11. Newton's Method root finder — $f(x) = x^3 + x - 3$.

```

program newton_root
  implicit none
  real :: x
  real, parameter :: error_tol = 0.000001
  integer :: i, maxit, converg

  x = 1.0
  maxit = 30
  i = 0
  converg = 0

```

```

do while (converg == 0 .and. i < maxit)
  x = x - f_x(x)/f_deriv_x(x)
  i = i + 1
  if (abs(f_x(x)) <= error_tol) converg = 1
end do

if (converg == 1) then
  write(*,*) 'Root is approximately', x
else
  write(*,*) 'Did not converge'
end if

contains
function f_x(x)
  real :: f_x, x
  f_x = x**3 + x - 3
end function f_x

function f_deriv_x(x)
  real :: f_deriv_x, x
  f_deriv_x = 3*x**2 + 1
end function f_deriv_x
end program newton_root

```

LEVEL 4 — Tricky, multi-step, "exam under time pressure" style

These combine string handling + numeric logic + loops — the style the midterm palindrome question tested.

Q12. PALINDROME CHECK — works for BOTH words and numbers (the midterm-style question)

Key trick: read the input as a **character string**, no matter whether the user types letters or digits. This way one piece of code handles "madam" and "12321" identically — you never need to know in advance which type it is.

```

program palindrome_check
  implicit none
  character(len=100) :: input_str

```

```

integer :: length, i
logical :: is_pal

write(*,*) 'Enter a word or a number:'
read(*,'(A)') input_str      ! read the WHOLE line as text

input_str = trim(adjustl(input_str)) ! remove leading/trailing blanks
length = len_trim(input_str)      ! actual length, ignoring blanks

is_pal = .true.
do i = 1, length/2
  if (input_str(i:i) /= input_str(length - i + 1:length - i + 1)) then
    is_pal = .false.
    exit
  end if
end do

if (is_pal) then
  write(*,*) trim(input_str), ' IS a palindrome'
else
  write(*,*) trim(input_str), ' is NOT a palindrome'
end if

end program palindrome_check

```

Why this is the tricky part of the exam:

- Fortran has **no built-in "reverse string" function** — you must manually compare character `i` from the front with character `(length-i+1)` from the back.
- `read(*, '(A)')` (character read) works for numbers-as-digits too, since digits are still valid characters — this is the trick that lets ONE program handle both cases, instead of writing separate integer logic and string logic.
- `length/2` with integer division automatically handles both even- and odd-length inputs correctly (middle character of an odd-length string never needs to be checked against itself).
- Common mistake: forgetting `trim(adjustl(...))`, which leaves trailing blanks in `input_str` and makes `length` wrong, causing correct palindromes to be falsely rejected.

Q13. ★ Palindrome check for a NUMBER using pure arithmetic (no strings)
— alternate approach

This is the version examiners sometimes want to see the "math-only" way, to test `mod`/integer-division skill.

```
program palindrome_number
  implicit none
  integer :: n, original, reversed, remainder

  write(*,*) 'Enter an integer:'
  read(*,*) n
  original = n
  reversed = 0

  do while (n /= 0)
    remainder = mod(n, 10)
    reversed = reversed * 10 + remainder
    n = n / 10
  end do

  if (reversed == original) then
    write(*,*) original, 'is a palindrome number'
  else
    write(*,*) original, 'is not a palindrome number'
  end if

end program palindrome_number
```

Logic explained: peel off the last digit with `mod(n, 10)`, build up a reversed number digit by digit, and shrink `n` with integer division `n/10` each pass. Compare the fully reversed number to the original at the end.

Q14. ★ Armstrong number check (digit-extraction trick, similar family to Q13)

An Armstrong number equals the sum of its own digits, each raised to the power of the digit count (e.g. $153 = 1^3 + 5^3 + 3^3$).

```
program armstrong_check
  implicit none
  integer :: n, original, digit, sum_val, num_digits, temp

  write(*,*) 'Enter a number:'
```

```

read(*,*) n
original = n

! Step 1: count digits
temp = n
num_digits = 0
do while (temp /= 0)
    num_digits = num_digits + 1
    temp = temp / 10
end do

! Step 2: sum of digit^num_digits
sum_val = 0
temp = n
do while (temp /= 0)
    digit = mod(temp, 10)
    sum_val = sum_val + digit**num_digits
    temp = temp / 10
end do

if (sum_val == original) then
    write(*,*) original, 'is an Armstrong number'
else
    write(*,*) original, 'is not an Armstrong number'
end if

end program armstrong_check

```

Q15. ★ Count vowels AND check if a sentence is a palindrome (ignoring spaces) — combines string trimming + array-style character indexing + counting

```

program vowel_and_palindrome
    implicit none
    character(len=100) :: raw, cleaned
    integer :: i, length, vowel_count
    logical :: is_pal
    character :: ch

    write(*,*) 'Enter a sentence:'
    read(*, '(A)') raw

```

```

! Build "cleaned" string with spaces removed
cleaned = "
do i = 1, len_trim(raw)
  ch = raw(i:i)
  if (ch /= ' ') then
    cleaned = trim(cleaned) // ch
  end if
end do

length = len_trim(cleaned)

! Count vowels
vowel_count = 0
do i = 1, length
  ch = cleaned(i:i)
  if (ch=='a' .or. ch=='e' .or. ch=='i' .or. ch=='o' .or. ch=='u' &
    .or. ch=='A' .or. ch=='E' .or. ch=='I' .or. ch=='O' .or. ch=='U') then
    vowel_count = vowel_count + 1
  end if
end do

! Check palindrome on the cleaned (space-free) string
is_pal = .true.
do i = 1, length/2
  if (cleaned(i:i) /= cleaned(length - i + 1:length - i + 1)) then
    is_pal = .false.
    exit
  end if
end do

write(*,*) 'Vowel count =', vowel_count
if (is_pal) then
  write(*,*) 'The cleaned sentence IS a palindrome'
else
  write(*,*) 'The cleaned sentence is NOT a palindrome'
end if

end program vowel_and_palindrome

```

Note the trick: `cleaned = trim(cleaned) // ch` — the `//` operator concatenates strings in Fortran. Building a string one character at a time like this is a common "harder" exam pattern (e.g., for "nab race car" → palindrome ignoring spaces).

Q16. ★ Matrix–vector multiplication with a dynamically-built identity matrix (array + nested loop + subroutine-style structure)

```
program identity_matrix_multiply
  implicit none
  real :: vec1(3), vec2(3), mtx_l(3,3)
  integer :: i, j

  vec1 = (/5.0, 5.0, 5.0/)

  ! Build identity matrix
  do i = 1, 3
    do j = 1, 3
      if (i == j) then
        mtx_l(i,j) = 1
      else
        mtx_l(i,j) = 0
      end if
    end do
  end do

  ! Matrix-vector product
  do i = 1, 3
    vec2(i) = 0
    do j = 1, 3
      vec2(i) = vec2(i) + mtx_l(i,j) * vec1(j)
    end do
  end do

  write(*,*) 'Result vector:', vec2

end program identity_matrix_multiply
```

Q17. ★ Find the second-largest number in an array without sorting (classic tricky pattern)

```
program second_largest
  implicit none
  integer :: arr(6), i, largest, second
  read(*,*) (arr(i), i = 1, 6)
```

```

if (arr(1) > arr(2)) then
    largest = arr(1)
    second = arr(2)
else
    largest = arr(2)
    second = arr(1)
end if

do i = 3, 6
    if (arr(i) > largest) then
        second = largest
        largest = arr(i)
    else if (arr(i) > second .and. arr(i) /= largest) then
        second = arr(i)
    end if
end do

write(*,*) 'Largest =', largest
write(*,*) 'Second largest =', second

end program second_largest

```

Watch out: initializing **largest/second** from the FIRST TWO elements (not just 0) is what trips people up if the array has negative numbers.

Q18. ★ Dynamic array + file writing + palindrome-style digit check combined (multi-part exam question)

Generate the first **n** Fibonacci numbers into a dynamically allocated array, write them to a file, and separately check whether the LAST Fibonacci number generated is itself a palindrome number.

```

program fibonacci_palindrome
    implicit none
    integer :: n, i
    integer, dimension(:), allocatable :: fib
    integer :: last_num, original, reversed, remainder
    logical :: is_pal

    write(*,*) 'How many Fibonacci numbers?'
    read(*,*) n

```

```

allocate(fib(n))

fib(1) = 0
if (n > 1) fib(2) = 1
do i = 3, n
    fib(i) = fib(i-1) + fib(i-2)
end do

! Write to file
open(10, file = 'fib.dat', status = 'replace')
do i = 1, n
    write(10,*) fib(i)
end do
close(10)

! Palindrome check on the last number generated
last_num = fib(n)
original = last_num
reversed = 0
do while (last_num /= 0)
    remainder = mod(last_num, 10)
    reversed = reversed * 10 + remainder
    last_num = last_num / 10
end do

is_pal = (reversed == original)

write(*,*) 'Last Fibonacci number:', original
if (is_pal) then
    write(*,*) 'It IS a palindrome number'
else
    write(*,*) 'It is NOT a palindrome number'
end if

deallocate(fib)

end program fibonacci_palindrome

```

PART 3 — QUICK "GOTCHA" CHECKLIST FOR THE EXAM

1. **Always** write `implicit none` first inside `program`.
2. Integer division truncates: $5/2 = 2$, not 2.5 . Use `real` variables if you need decimals.
3. `mod(a,b)` gives the remainder — the backbone of palindrome/digit/even-odd tricks.
4. Fortran arrays default to starting index **1**, not **0** — unless you explicitly declare a custom range like `dimension(-5:0,0:5)` or `dimension(0:n)`.
5. String comparison uses `character(i:i)` slicing — there is no built-in `reverse()` function.
6. `//` is the string concatenation operator.
7. `trim()` removes trailing blanks; `adjustl()` removes leading blanks; use both together when reading user text.
8. `exit` breaks a `do` loop early; there is no `break` keyword.
9. In a **function**, the return value is assigned to the function's own name (`cal_avg = sum/3`), not to a `return` variable.
10. In a **subroutine**, you `call` it, and it changes its arguments directly (no return value).
11. `intent(in)` protects an argument from being modified inside a function/subroutine — leaving it out is a classic "what's the bug" trick question.
12. Fortran is **not case-sensitive** — `Total` and `total` are the same variable (this can hide bugs where you think you made a new variable).
13. For dynamic memory: `allocate()` before use, `deallocate()` when done — otherwise it's a memory leak.
14. `where/elsewhere/end where` lets you apply conditions to a WHOLE array without a loop.

Fortran 95 Practice Question Bank

CSE 425 – Concepts of Programming Languages

Covers: declarations, data types, operators, I/O, if-else, do loops, arrays (static/dynamic), functions (internal/external), subroutines, root-finding, and file I/O — from basic warm-ups to exam-style, time-pressure "gotcha" problems.

Each question is followed immediately by a full working answer. Questions get progressively harder within each section, and the last section mixes everything together the way a tricky exam question would.

SECTION 1 — Basics: Declarations, Data Types, Operators

Q1 (Easy).

Write a program that declares two integers and two reals, reads them from the user, and prints their sum, difference, product, and (real) quotient. Use `implicit none` and the `::` separator.

Answer

! Q1: basic declarations and arithmetic

```
program basic_arith
  implicit none
  integer :: i1, i2
  real    :: r1, r2

  print *, 'Enter two integers:'
  read *, i1, i2
  print *, 'Enter two real numbers:'
  read *, r1, r2

  write(*,*) 'Integer sum      =', i1 + i2
  write(*,*) 'Integer difference =', i1 - i2
  write(*,*) 'Real product     =', r1 * r2
  write(*,*) 'Real quotient    =', r1 / r2
end program basic_arith
```

Key idea: `::` is required once you mix attributes (or just for good style); `implicit none` forces you to declare every variable explicitly instead of relying on the i–n integer / a–h,o–z real naming convention.

Q2 (Easy–Medium, classic "gotcha").

Without using a temporary/dummy variable, swap the values of two integers **a** and **b** using pure arithmetic. Then explain in one line **why** this is risky for real numbers.

Answer

```
program swap_no_temp
  implicit none
  integer :: a, b
  a = 9
  b = 4
  a = a + b    ! a = 13
  b = a - b    ! b = 13 - 4 = 9 (original a)
  a = a - b    ! a = 13 - 9 = 4 (original b)
  write(*,*) 'a =', a, ' b =', b
end program swap_no_temp
```

Gotcha note: with **real** types this can silently **overflow** or lose precision for very large/small numbers, and it fails outright for **complex/character** types where **+/-** aren't meaningfully invertible the same way. In an exam, the "safe" general-purpose answer is always the 3-line temp-variable swap — only use the arithmetic trick when explicitly asked to avoid a temp variable.

Q3 (Easy).

Declare **pi** as a named constant (**parameter**) and write a program that reads a radius and prints the circle's area and circumference.

Answer

```
program circle_calc
  implicit none
  real, parameter :: pi = 3.1416
  real :: radius, area, circumference

  print *, 'Enter radius:'
  read *, radius
  area = pi * radius**2
  circumference = 2.0 * pi * radius

  write(*,*) 'Area      =', area
  write(*,*) 'Circumference =', circumference
end program circle_calc
```

Q4 (Medium, tricky — precision).

Fortran's default `real` gives about 6–7 significant digits. Declare a **double-precision** real using `SELECTED_REAL_KIND`, and use it to compute `1.0/3.0` printed to high precision. Compare it (in a comment) with the default single-precision result.

Answer

```
program precision_demo
  implicit none
  integer, parameter :: dp = selected_real_kind(15, 300) ! ~15 digits, range 10^300
  real                :: single_val
  real(kind=dp)       :: double_val

  single_val = 1.0 / 3.0
  double_val = 1.0_dp / 3.0_dp ! the _dp suffix forces the LITERAL to be double too

  write(*,*) 'single precision :', single_val
  write(*,*) 'double precision :', double_val
end program precision_demo
```

Gotcha: writing `double_val = 1.0/3.0` (without the `_dp` suffix) computes the division in **single precision first** and then just stores the truncated result in the double variable — a classic exam trap. You must attach the kind suffix to the literals themselves.

Q5 (Medium).

Read a person's name (`character`) and age (`integer`), then print a formatted greeting sentence using `write`. Trim any trailing blanks from the name.

Answer

```
program greet
  implicit none
  character(len=30) :: name
  integer :: age

  print *, 'Enter your name:'
  read *, name
```

```
print *, 'Enter your age:'  
read *, age
```

```
write(*,*) 'Hello ', trim(name), ', you are ', age, ' years old.'  
end program greet
```

Note: `character(len=30) :: name` reserves 30 characters padded with blanks; `trim()` strips the trailing blanks so the concatenation with `', you are '` looks clean.

SECTION 2 — I/O and Logical Control (if / else if / nested if)

Q6 (Easy–Medium).

Classify a student's score into A / A- / B+ / B / F using `if ... else if ... end if` (F if score < 60, in addition to the grades already shown in the slide logic).

Answer

```
program lettergrade2  
  implicit none  
  real :: score  
  print *, 'Enter CSE 425 score'  
  read *, score  
  
  if (score >= 93) then  
    write(*,*) 'Letter grade is A'  
  else if (score >= 90 .and. score < 93) then  
    write(*,*) 'Letter grade is A-'  
  else if (score >= 87 .and. score < 90) then  
    write(*,*) 'Letter grade is B+'  
  else if (score >= 60 .and. score < 87) then  
    write(*,*) 'Letter grade is B'  
  else  
    write(*,*) 'Letter grade is F'  
  end if  
end program lettergrade2
```

Q7 (Medium, tricky boundary logic).

Write a program to check whether a given year is a leap year. Remember the real rule: divisible by 4 **and** (not divisible by 100 **or** divisible by 400).

Answer

```
program leap_year
  implicit none
  integer :: year
  logical :: is_leap

  print *, 'Enter a year:'
  read *, year

  is_leap = (mod(year,4) == 0 .and. mod(year,100) /= 0) .or. (mod(year,400) == 0)

  if (is_leap) then
    write(*,*) year, 'is a leap year'
  else
    write(*,*) year, 'is NOT a leap year'
  end if
end program leap_year
```

Gotcha: students often write `mod(year,4)==0 .and. mod(year,100)/=0 .or. mod(year,400)==0` **without parentheses**. Because `.and.` binds tighter than `.or.` this actually evaluates correctly by luck here, but relying on operator precedence instead of explicit parentheses is a common source of subtle logic bugs — always parenthesize compound boolean expressions.

Q8 (Medium–Hard, nested if / labeled blocks).

Using **labeled** nested `if` blocks (`outer:inner:`, as in the slide's square-root example), write a program that reads two integers `a` and `b` and safely computes `a/b`. If `b = 0`, print an error. If `a/b` would be negative, print a warning but still show the value.

Answer

```
program safe_divide
  implicit none
  integer :: a, b
  real    :: result
```

```

print *, 'Enter a and b:'
read *, a, b

outer: if (b /= 0) then
    result = real(a) / real(b)
    inner: if (result < 0) then
        write(*,*) 'Warning: result is negative ->', result
    else inner
        write(*,*) 'Result =', result
    end if inner
else outer
    write(*,*) 'Error: division by zero'
end if outer
end program safe_divide

```

Key idea: labeling `if` blocks (`outer:`, `inner:`) is purely for readability in large nested structures and lets you close each block explicitly with `end if outer` / `end if inner`, avoiding ambiguity about which `end if` matches which `if`.

SECTION 3 — Repetition: `do` loops

Q9 (Easy).

Print all even numbers from 2 to 50 using a `do` loop with a step.

Answer

```

program even_numbers
    implicit none
    integer :: i
    do i = 2, 50, 2
        write(*,*) i
    end do
end program even_numbers

```

Q10 (Medium, tricky `exit` usage).

Read a positive integer `n` and determine if it's **prime**, by looping only up to `sqrt(n)` and using `exit` as soon as a divisor is found.

Answer

```
program is_prime
  implicit none
  integer :: n, i
  logical :: prime

  print *, 'Enter a positive integer:'
  read *, n

  prime = .true.
  if (n < 2) prime = .false.

  i = 2
  do while (i*i <= n .and. prime)
    if (mod(n, i) == 0) then
      prime = .false.
      exit      ! no need to keep checking once we found a factor
    end if
    i = i + 1
  end do

  if (prime) then
    write(*,*) n, 'is prime'
  else
    write(*,*) n, 'is NOT prime'
  end if
end program is_prime
```

Gotcha: looping `i` all the way to `n` (instead of `sqrt(n)`) still works but is needlessly slow for large `n` — an exam question may specifically ask you to justify the `i*i <= n` bound instead of computing `sqrt(n)` (which involves a real-valued function call and possible rounding issues).

Q11 (Medium–Hard, nested loops + pattern).

Print the following right-angled triangle of stars for a user-given height `n` (e.g., `n=4` gives 4 rows: `*`, `**`, `***`, `****`) **without** using any string-repeat intrinsic — pure nested loops only.

Answer

```
program star_triangle
  implicit none
  integer :: n, i, j

  print *, 'Enter height:'
  read *, n

  do i = 1, n
    do j = 1, i
      write(*,'(A)', advance='no') '*'
    end do
    write(*,*)           ! move to next line
  end do
end program star_triangle
```

Key idea: `advance='no'` in the `write` statement suppresses the automatic newline so stars print on the same line; a bare `write(*,*)` with no list is used just to force the newline at the end of each row.

Q12 (Hard, do-while with multiple exit conditions — classic time-crunch trap).

Simulate a simple number-guessing check: given a target number `target` and a fixed sequence of guesses stored in an array `guesses(5)`, loop through the guesses and stop **either** when the guess matches the target **or** after all 5 guesses are used, printing how many attempts it took (or "not found").

Answer

```
program guess_check
  implicit none
  integer :: target, guesses(5), i, attempts
  logical :: found

  target = 42
  guesses = (/ 10, 25, 42, 7, 99 /) ! array constructor

  found = .false.
  attempts = 0
```

```

do i = 1, 5
  attempts = attempts + 1
  if (guesses(i) == target) then
    found = .true.
    exit
  end if
end do

if (found) then
  write(*,*) 'Found target in', attempts, 'attempts'
else
  write(*,*) 'Target not found after', attempts, 'attempts'
end if
end program guess_check

```

Gotcha: if you forget the `exit`, `attempts` will keep incrementing to 5 even after finding the match on attempt 3, giving a wrong "attempts" count — a very common off-by-logic mistake under time pressure.

SECTION 4 — Arrays (static, 2-D, and dynamic/allocatable)

Q13 (Easy–Medium).

Read 5 real numbers into an array and print the maximum, minimum, and average — without using the `maxval/minval` intrinsics (write the loop logic manually).

Answer

```

program array_stats
  implicit none
  real :: arr(5), maxv, minv, total, avg
  integer :: i

  print *, 'Enter 5 numbers:'
  read *, (arr(i), i = 1, 5)

  maxv = arr(1); minv = arr(1); total = 0.0
  do i = 1, 5

```

```

        if (arr(i) > maxv) maxv = arr(i)
        if (arr(i) < minv) minv = arr(i)
        total = total + arr(i)
    end do
    avg = total / 5.0

    write(*,*) 'Max =', maxv, ' Min =', minv, ' Avg =', avg
end program array_stats

```

Q14 (Medium, in-place reversal — pointer/index trickiness).

Reverse an array of size **n** **in place** (do not use a second array), using a **do** loop that only goes halfway through the array.

Answer

```

program reverse_array
    implicit none
    integer, parameter :: n = 6
    integer :: arr(n), i, temp

    arr = (/ 1, 2, 3, 4, 5, 6 /)

    do i = 1, n/2
        temp      = arr(i)
        arr(i)    = arr(n - i + 1)
        arr(n-i+1) = temp
    end do

    write(*,*) 'Reversed array:', arr
end program reverse_array

```

Gotcha: looping **i = 1, n** instead of **i = 1, n/2** would swap every pair **twice**, silently restoring the original (unreversed) order — a subtle bug that "looks like it works" if you only check that the program runs without crashing.

Q15 (Medium–Hard, 2-D arrays).

Given two 3×3 matrices **A** and **B**, compute their sum $C = A + B$ and also their element-wise product (Hadamard product), printing both matrices row by row.

Answer

```
program matrix_ops
  implicit none
  integer :: A(3,3), B(3,3), C_sum(3,3), C_prod(3,3), i, j

  A = reshape( (/1,2,3, 4,5,6, 7,8,9/), (/3,3/) )
  B = reshape( (/9,8,7, 6,5,4, 3,2,1/), (/3,3/) )

  C_sum = A + B      ! whole-array arithmetic, no explicit loop needed
  C_prod = A * B      ! element-wise, NOT matrix multiplication

  write(*,*) 'Sum matrix:'
  do i = 1, 3
    write(*,*) (C_sum(i,j), j = 1, 3)
  end do

  write(*,*) 'Element-wise product matrix:'
  do i = 1, 3
    write(*,*) (C_prod(i,j), j = 1, 3)
  end do
end program matrix_ops
```

Gotcha: `reshape( (/1,2,3,4,5,6,7,8,9/), (/3,3/) )` fills the matrix **column-major** (Fortran's native storage order), so **A** is actually:

```
1 4 7
2 5 8
3 6 9
```

not the row-major layout you might expect from reading the list left to right — this trips up almost everyone the first time.

Q16 (Hard, real matrix multiplication — distinguishing from Q15).

Now compute the **true matrix product** $A \times B$ for two 3×3 matrices (i.e., $C(i,j) = \sum_k A(i,k) * B(k,j)$), without using the `matmul` intrinsic.

Answer

```
program matrix_multiply
  implicit none
  integer :: A(3,3), B(3,3), C(3,3), i, j, k

  A = reshape( (/1,4,7, 2,5,8, 3,6,9/), (/3,3/) ) ! gives row-major-looking 1 2 3 / 4 5 6 / 7 8 9
  B = reshape( (/1,0,0, 0,1,0, 0,0,1/), (/3,3/) ) ! identity

  C = 0
  do i = 1, 3
    do j = 1, 3
      do k = 1, 3
        C(i,j) = C(i,j) + A(i,k) * B(k,j)
      end do
    end do
  end do

  write(*,*) 'Product matrix C = A x B:'
  do i = 1, 3
    write(*,*) (C(i,j), j = 1, 3)
  end do
end program matrix_multiply
```

Note: Because Fortran is column-major, feeding **reshape** the sequence **1, 4, 7, 2, 5, 8, 3, 6, 9** produces the matrix that *looks* like **1 2 3 / 4 5 6 / 7 8 9** when printed row by row — this is the counter-trick to Q15's gotcha and is a favorite "predict the output" exam question.

Q17 (Hard, dynamic/allocatable arrays).

Write a program that first asks the user how many numbers they want to enter, **allocates** an array of exactly that size, reads the values, computes the mean and the (population) standard deviation, then **deallocates** the array.

Answer

```
program dynamic_stats
  implicit none
  real, dimension(:), allocatable :: data
  integer :: n, i
  real :: mean, variance, std_dev, sum_val, sum_sq
```

```

print *, 'How many numbers?'
read *, n
allocate(data(n))

print *, 'Enter the numbers:'
read *, (data(i), i = 1, n)

sum_val = 0.0
do i = 1, n
    sum_val = sum_val + data(i)
end do
mean = sum_val / n

sum_sq = 0.0
do i = 1, n
    sum_sq = sum_sq + (data(i) - mean)**2
end do
variance = sum_sq / n
std_dev = sqrt(variance)

write(*,*) 'Mean      =', mean
write(*,*) 'Standard deviation =', std_dev

deallocate(data)
end program dynamic_stats

```

Gotcha: forgetting to `allocate(data(n))` before using `data` results in a run-time crash (the array is not usable until allocated) — unlike static arrays declared with a fixed size, `allocatable` arrays start out with **no** storage at all.

SECTION 5 — Functions (external & internal) and Recursion

Q18 (Medium).

Write an **external** function `is_even(x)` (logical) and use it inside the main program to print all even numbers from an array.

Answer

```

program filter_even
  implicit none
  integer :: arr(6), i
  logical :: is_even

  arr = (/3, 8, 15, 22, 41, 56/)

  do i = 1, 6
    if (is_even(arr(i))) write(*,*) arr(i), 'is even'
  end do
end program filter_even

```

```

logical function is_even(x)
  implicit none
  integer, intent(in) :: x
  is_even = (mod(x,2) == 0)
end function is_even

```

Note: because `is_even` is **external** (defined after `end program`, not inside a `contains` block), the calling program must declare its type (`logical :: is_even`) itself — the compiler has no other way to know its return type.

Q19 (Medium–Hard, internal function with `contains`).

Rewrite Q18 so `is_even` is an **internal** function using `contains`, and explain the one practical advantage this has over the external version.

Answer

```

program filter_even_internal
  implicit none
  integer :: arr(6), i

  arr = (/3, 8, 15, 22, 41, 56/)

  do i = 1, 6
    if (is_even(arr(i))) write(*,*) arr(i), 'is even'
  end do

contains

```

```

logical function is_even(x)
  implicit none
  integer, intent(in) :: x
  is_even = (mod(x,2) == 0)
end function is_even

```

```

end program filter_even_internal

```

Advantage: internal functions **automatically inherit the interface** (return type, argument types) from being compiled together with the host program, so you never need a separate type declaration for the function name in the caller — the compiler checks argument types for you and catches mismatches at compile time, which an external function (compiled separately) cannot always do.

Q20 (Hard, recursion — a classic trick topic).

Write a **recursive** function to compute $n!$ (factorial), and trace by hand (in a comment) what happens if someone accidentally forgets the **recursive** keyword.

Answer

```

program recursive_factorial
  implicit none
  integer :: n, result

  print *, 'Enter n:'
  read *, n
  result = factorial(n)
  write(*,*) n, '!=', result

```

contains

```

recursive function factorial(n) result(fact_val)
  implicit none
  integer, intent(in) :: n
  integer :: fact_val
  if (n <= 1) then
    fact_val = 1
  else
    fact_val = n * factorial(n - 1)
  end if
end function factorial

```

```
end program recursive_factorial
```

Gotcha: if you omit the `recursive` keyword, most compilers will **reject the compile** with an error like "recursive call to non-recursive function," because by default Fortran functions are not re-entrant (the compiler may allocate their local storage in a way that doesn't support calling themselves). Also note the `result(fact_val)` clause: it's used here so the function name `factorial` itself can still be called recursively inside its own body without ambiguity — trying to assign directly to `factorial = n*factorial(n-1)` gets confusing and is best avoided in recursive functions.

SECTION 6 — Subroutines and `intent`

Q21 (Medium).

Write a subroutine `stats(arr, n, avg, maxv)` that takes an array and its size as input and returns (via arguments) the average and maximum. Use correct `intent` attributes.

Answer

```
program call_stats
  implicit none
  real :: numbers(5), average, maximum

  numbers = (/12.0, 45.0, 3.0, 78.0, 29.0/)
  call stats(numbers, 5, average, maximum)
```

```
  write(*,*) 'Average =', average
  write(*,*) 'Maximum =', maximum
```

contains

```
  subroutine stats(arr, n, avg, maxv)
    implicit none
    integer, intent(in) :: n
    real,   intent(in)  :: arr(n)
    real,   intent(out) :: avg, maxv
    integer :: i

    maxv = arr(1)
```

```

    avg = 0.0
    do i = 1, n
        avg = avg + arr(i)
        if (arr(i) > maxv) maxv = arr(i)
    end do
    avg = avg / n
end subroutine stats

```

end program call_stats

Key idea: `intent(in)` = read-only input, `intent(out)` = the subroutine must set it (its incoming value is undefined/ignored), `intent(inout)` = both read and modified. Getting these wrong doesn't always cause a compile error, but it removes the compiler's ability to catch you accidentally overwriting an input you meant to only read.

Q22 (Hard, `intent(inout)` + array modification trap).

Write a subroutine `double_positive(arr, n)` that **modifies the array in place**, doubling only the positive elements and leaving negatives/zero untouched. Then explain what would go wrong if you declared the array argument as `intent(in)` instead.

Answer

```

program modify_array
    implicit none
    integer :: arr(6), i

    arr = (/ -3, 5, 0, 8, -1, 2 /)

    call double_positive(arr, 6)

    write(*,*) 'Modified array:', (arr(i), i = 1, 6)

```

contains

```

subroutine double_positive(arr, n)
    implicit none
    integer, intent(in) :: n
    integer, intent(inout) :: arr(n)
    integer :: i
    do i = 1, n

```

```

        if (arr(i) > 0) arr(i) = arr(i) * 2
    end do
end subroutine double_positive

```

```

end program modify_array

```

Gotcha: if `arr` were declared `intent(in)`, the line `arr(i) = arr(i) * 2` inside the subroutine would be a **compile-time error** (you cannot assign to an `intent(in)` dummy argument) — this is exactly the kind of thing an exam question tests to see if you understand that `intent` is enforced by the compiler, not just documentation.

Q23 (Hard, sorting via subroutine — full time-crunch style problem).

Write a subroutine `bubble_sort(arr, n)` that sorts an integer array in ascending order in place, and call it from the main program on a 6-element array, printing before and after.

Answer

```

program sort_demo
    implicit none
    integer :: arr(6) = (/ 34, 7, 23, 32, 5, 62 /)
    integer :: i

    write(*,*) 'Before:', arr
    call bubble_sort(arr, 6)
    write(*,*) 'After :', arr

```

contains

```

subroutine bubble_sort(arr, n)
    implicit none
    integer, intent(in) :: n
    integer, intent(inout) :: arr(n)
    integer :: i, j, temp

    do i = 1, n - 1
        do j = 1, n - i
            if (arr(j) > arr(j+1)) then
                temp = arr(j)
                arr(j) = arr(j+1)
                arr(j+1) = temp
            end if
        end do
    end do

```

```

        end if
    end do
end do
end subroutine bubble_sort

end program sort_demo

```

Gotcha (very common): the inner loop bound must be $n - i$, **not** $n - 1$ for every pass — otherwise you either access `arr(n+1)` (out-of-bounds, undefined behavior/crash) or you waste comparisons re-checking already-sorted tail elements. Also note the array is given an initializer directly in its declaration (`= (/.../)`), which implicitly gives it the `save` attribute in some contexts — safe here since it's a simple local array in the main program, but worth knowing for functions/subroutines called repeatedly.

SECTION 7 — Mixed / Toughest Exam-Style Problems

Q24 (Hard, Newton–Raphson variant).

Adapt the Newton's Method pattern from the slides to find a root of $f(x) = x^2 - 2$ (i.e., approximate $\sqrt{2}$), starting from $x = 1$, iterating until $|f(x)| \leq 1e-6$ or a maximum of 50 iterations, and reporting whether it converged.

Answer

```

program newton_sqrt2
  implicit none
  real :: x = 1.0
  real, parameter :: error_tol = 1.0e-6
  integer :: i, maxit, converg

  maxit = 50
  converg = 0
  i = 0

  do while (converg == 0 .and. i < maxit)
    x = x - f_x(x) / f_deriv_x(x)
    i = i + 1
    if (abs(f_x(x)) <= error_tol) converg = 1
  end do

```

```

if (converg == 1) then
  write(*,*) 'Converged after', i, 'iterations. Root =', x
else
  write(*,*) 'Did not converge within', maxit, 'iterations.'
end if

```

contains

```

function f_x(x)
  real :: f_x, x
  f_x = x**2 - 2.0
end function f_x

function f_deriv_x(x)
  real :: f_deriv_x, x
  f_deriv_x = 2.0 * x
end function f_deriv_x

```

end program newton_sqrt2

Gotcha: if the starting guess $x = 0$ were used, $f_deriv_x(0) = 0$ and the update $x - f_x(x)/f_deriv_x(x)$ would **divide by zero** — a good exam question is "for which starting values does this Newton iteration fail, and why?" The answer: any x where the derivative is zero, since the iteration formula has it in the denominator.

Q25 (Hard, file I/O + arrays combined — full mini-project style, matches the "Data Generation and Storing" slide).

Write a program that:

1. Asks the user for n (number of points),
2. Computes $y = \sin(x)$ for $x = 0, 0.1, 0.2, \dots$ up to n points using a dynamic array,
3. Writes the (x, y) pairs to a file `sine_data.dat` (suitable for plotting with `gnuplot` as in the slides),
4. Reads the same values back from the file into new arrays and prints the maximum y value found.

Answer

```

program sine_data_generate

```

```

implicit none
integer :: n, i, unit_id
real :: dx
real, dimension(:), allocatable :: x_arr, y_arr
real, dimension(:), allocatable :: x_check, y_check
real :: max_y

print *, 'Enter number of points:'
read *, n

allocate(x_arr(n), y_arr(n))
dx = 0.1

do i = 1, n
    x_arr(i) = (i - 1) * dx
    y_arr(i) = sin(x_arr(i))
end do

! --- write to file ---
unit_id = 12
open(unit_id, file = 'sine_data.dat', status = 'replace')
do i = 1, n
    write(unit_id, *) x_arr(i), y_arr(i)
end do
close(unit_id)

! --- read back from file to verify ---
allocate(x_check(n), y_check(n))
open(unit_id, file = 'sine_data.dat', status = 'old')
do i = 1, n
    read(unit_id, *) x_check(i), y_check(i)
end do
close(unit_id)

max_y = y_check(1)
do i = 2, n
    if (y_check(i) > max_y) max_y = y_check(i)
end do

write(*,*) 'Data written to sine_data.dat'
write(*,*) 'Maximum y value read back =', max_y

deallocate(x_arr, y_arr, x_check, y_check)
end program sine_data_generate

```

Gnuplot follow-up (as in the slides):

```
gnuplot> plot 'sine_data.dat' using 1:2 with lines title "sin(x)"
gnuplot> set xlabel "x"
gnuplot> set ylabel "sin(x)"
```

Gotcha: `status = 'new'` (as shown in the slide example) will **fail** if `sine_data.dat` already exists from a previous run — for a program you intend to re-run repeatedly during testing, `status = 'replace'` (overwrite if it exists, create if it doesn't) is the safer choice, whereas `status = 'old'` when *reading* requires the file to already exist.

Q26 (Toughest, "predict the output" style — combines arrays, functions, and column-major order).

Without running it, determine what the following program prints, then explain why.

```
program predict_me
  implicit none
  real :: M(2,3)
  integer :: i, j

  M = reshape( (/1.0, 2.0, 3.0, 4.0, 5.0, 6.0/), (/2,3/) )

  do i = 1, 2
    write(*,*) (M(i,j), j = 1, 3)
  end do

  write(*,*) 'Sum via function:', total(M, 2, 3)
```

contains

```
function total(A, r, c)
  real :: total
  integer, intent(in) :: r, c
  real, intent(in) :: A(r,c)
  integer :: p, q
  total = 0.0
  do p = 1, r
    do q = 1, c
```

```
        total = total + A(p,q)
    end do
end do
end function total
```

```
end program predict_me
```

Answer / Explanation `reshape` fills column-major, so a 2×3 array from the flat list `1, 2, 3, 4, 5, 6` becomes:

```
Row 1: 1.0  3.0  5.0
Row 2: 2.0  4.0  6.0
```

So the printed output is:

```
1.0  3.0  5.0
2.0  4.0  6.0
Sum via function:  21.0
```

The sum is unaffected by row/column order ($21.0 = 1+2+3+4+5+6$ regardless of layout), but the **row-by-row printout** absolutely depends on understanding column-major fill order — this is the single most common "trick question" involving Fortran arrays, precisely because Fortran's storage order is the opposite of C/Python, which is what most students are used to.

Quick Self-Check Checklist (use before an exam)

- ☐ Can I state the difference between `intent(in)`, `intent(out)`, `intent(inout)` and what breaks if misused?
- ☐ Do I know Fortran arrays/`reshape` fill **column-major**, not row-major?
- ☐ Can I write a `do` loop with a `step`, and use `exit` correctly (without double-counting)?
- ☐ Do I know when to use `allocate/deallocate` vs. a fixed-size array?
- ☐ Can I write both an **external** function (declared in the caller) and an **internal** function (`contains`)?
- ☐ Do I know why `recursive` is mandatory for self-calling functions?
- ☐ Can I open/write/read/close a file with the correct `status` (`'new'`, `'old'`, `'replace'`)?
- ☐ Can I trace a Newton's Method loop by hand, including why a zero derivative breaks it?

Good clarification. Let me reason about **what kind of question the palindrome check actually is**, then match that pattern across your other topics.

Pattern of the midterm question:

- Single self-contained algorithm (not multi-part)
- Input: one variable (a string) from the user
- Core logic: **one loop + one conditional check**, ~15–25 lines total
- Tests whether you can correctly set loop bounds and index into a data structure (`str(i:i)` vs `str(length-i+1:...)`)
- Has a natural "off-by-one" trap built in
- Outputs a simple yes/no or Boolean-style result

That's a very specific complexity tier: **not** trivial (like "print sum of two numbers") and **not** a multi-function mini-project (like Q25/Q26 in the bank). It's the "one clean algorithm, one tricky indexing detail" tier. Finals tend to repeat this *exact* tier once per major topic covered since the midterm, so expect roughly one question like this per bullet below.

Predicted final questions, matched to that complexity tier

1. Arrays — "is it sorted / has a duplicate" Same shape as palindrome: one loop, one comparison, boolean result.

! Check if array is sorted in ascending order

```
logical :: sorted
```

```
sorted = .true.
```

```
do i = 1, n-1
```

```
  if (arr(i) > arr(i+1)) then
```

```
    sorted = .false.
```

```
    exit
```

```
  end if
```

```
end do
```

Equally likely: "check if an array contains any duplicate values" (nested loop, same tier but $O(n^2)$).

2. 2-D arrays — "is the matrix symmetric" Direct array analogue of palindrome (compare `M(i,j)` to `M(j,i)` instead of `str(i)` to `str(n-i+1)`).

```
logical :: symmetric
```

```
symmetric = .true.
```

```
do i = 1, n
```

```
do j = 1, n
  if (M(i,j) /= M(j,i)) symmetric = .false.
end do
end do
```

3. Functions — "is prime" or "GCD via Euclid's algorithm" Single self-contained function, one loop, one conditional — same size as palindrome but wrapped as a **function** instead of inline code (tests the function-syntax topic on top of the same algorithmic complexity).

4. Subroutines — same problem, but "rewrite as a subroutine with **intent"** Very likely they take *last term's* algorithm (or a close cousin) and ask you to express it as a subroutine with **intent(in)/intent(out)**, since that's the natural progression after teaching subroutines post-midterm.

5. Recursion — "reverse a string recursively" or "Fibonacci(n)" Recursion is usually tested as a direct reformulation of an iterative pattern you already know (palindrome's reversal-comparison idea maps naturally onto a recursive reverse-and-compare).

6. Do loops / nested loops — "count vowels" or "check anagram of two strings" Anagram-checking is the most likely direct sibling of palindrome: same string-indexing skill, one loop level up in complexity (needs a frequency count or sort, ~20–30 lines).

7. File I/O — "read numbers from a file, check if they're a palindrome sequence" or "count how many lines/values satisfy some condition" Since file I/O appeared in your slides but likely *after* the midterm, finals often bolt it onto a familiar algorithm: same core logic as palindrome, but data comes from **read(unit,*)** inside a loop instead of the keyboard.

8. Root-finding / numerical — "Newton's method for a different function" or "bisection method" This is the "big" question tier (like Q24) — more likely to be a *separate*, harder question rather than matching palindrome's tier, since it's clearly a distinct topic block in your slides.

My best guess at final structure

Given the pattern, a realistic final probably has:

- 2–3 questions at the **palindrome tier** (one loop + one conditional, on arrays/strings) — e.g., anagram check, symmetric matrix, sorted-array check
- 1–2 questions requiring a **function or subroutine wrapper** around a similarly-sized algorithm
- 1 **recursion** question (GCD or Fibonacci, since factorial/fib are the standard teaching examples)

- 1 **file I/O** question, likely combined with an array/loop task
- Possibly 1 heavier **Newton's method / numerical** question as the "hardest" one, separate in style and length

Want me to add fully worked versions of these predicted questions (anagram check, symmetric matrix, GCD via subroutine and via recursion, sorted-array check, file-based counting) into your practice bank file so you have them ready the same way as the others?